



WEEKLY STUDENT INTERNSHIP JOURNAL

STUDENT INTERN: Joshua D. Barcoma	PROGRAM: BSIT
DATE: Feb 19-25, 2026	

WEEK 6

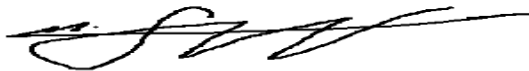
ACCOMPLISHMENT	DIFFICULTIES ENCOUNTERED	NEW LEARNING/ INSIGHTS
Development of Multi-Tenant Memory Storage System - Implemented PostgreSQL with pgvector for vector search - Created secure multi-tenant storage isolation - Developed storage key authentication system - Automated database schema initialization	Implementing a secure and scalable multi-tenant storage system presented the primary technical challenge. Each storage entity needed to maintain its own distinct set of memories with complete isolation from other storage entities, requiring meticulous design of authentication mechanisms and database schema structures. Balancing security concerns with system performance proved complex, as each request required validation and routing to the appropriate isolated storage space.	This project provided extensive learning opportunities in modern AI agent architecture, particularly function calling capabilities. Implementing OpenAI's function calling feature revealed how AI systems can dynamically invoke external tools based on natural language commands, transforming AI agents into sophisticated assistants capable of executing real-world actions.
Vector Embedding Integration - Integrated OpenAI embeddings (text-embedding-3-small, 1536D) - Supported configurable embedding dimensions - Implemented cosine similarity search with IVFFLAT - Built extensible embedding support (OpenAI & SentenceTransformers)	Optimizing vector similarity search performance utilizing the pgvector extension encountered significant challenges. Initial implementations demonstrated suboptimal query performance as the volume of stored memories increased. Configuring the IVFFLAT index required meticulous parameter tuning to balance search accuracy and query execution speed. Understanding vector indexing algorithms proved essential, as initial configurations that worked for small datasets became inefficient as the dataset grew, requiring recalibration and sometimes complete index reconstruction.	Working with the pgvector extension provided comprehensive insights into vector database technologies and semantic search methodologies. Vector embeddings effectively capture semantic meaning and contextual relationships, enabling the system to identify relevant memories even when exact lexical matches are absent. Understanding how embedding dimensions, normalization techniques, and indexing strategies impact search accuracy and performance proved essential for constructing modern AI applications.
Buddy AI with Persistent Memory - Developed AI assistant with persistent memory - Automated memory extraction and storage - Implemented semantic memory search and retrieval - Enabled context-aware responses using conversation history	Developing intelligent memory extraction, editing, and deletion logic presented considerable complexity. The system needed to automatically detect when to save, update, or delete memories based on natural language user requests expressed in numerous ways. This functionality necessitated multiple LLM API calls and sophisticated decision-making logic to interpret user intent accurately. Handling ambiguous requests proved particularly challenging, where the system must determine whether a user statement represents new information, an update, or a deletion request.	Implementing storage-based isolation mechanisms offered valuable learning experiences regarding multi-tenant architecture patterns and security considerations. This experience provided important lessons concerning data separation strategies, authentication methodologies, and architectural approaches for designing systems capable of scaling to support multiple isolated tenants. Utilizing large language models for structured decision-making beyond simple text generation revealed significant insights. The system employs multiple LLM API calls to make decisions regarding memory operations, demonstrating how LLMs can function as intelligent decision engines when properly structured and prompted. This approach illuminated both the capabilities and limitations of LLM-



Republic of the Philippines
Pamantasan ng Cabuyao
(University of Cabuyao)
Academic Affairs Division

Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

Memory Management Features • Enabled user-triggered memory deletion • Implemented memory editing and updating • Added LLM-based intelligent memory extraction • Supported deletion via keyword, ID, and contextual filtering LibreChat Integration • Integrated Buddy AI with LibreChat frontend/backend • Configured CORS middleware • Developed OpenAI-compatible chat endpoint • Established service-to-service connectivity API Development • Built RESTful FastAPI endpoints • Implemented authenticated storage management • Developed Buddy chat endpoint with error handling • Added input validation and robust error management Application Containerization (Docker) • Docker Image Creation • Running Application Using Docker		based logic, necessitating careful prompt engineering and fallback mechanisms. Integrating with LibreChat provided valuable insights into connecting custom AI services with existing chat platforms. This integration required understanding LibreChat's architecture, configuring CORS middleware for cross-origin requests, and developing an OpenAI-compatible chat completion endpoint. The experience highlighted the importance of API compatibility standards and how to adapt custom services to work with existing platform requirements. Working with FastAPI framework provided comprehensive experience with contemporary Python web development practices, including dependency injection patterns, Pydantic models for data validation, asynchronous programming, and RESTful API design. FastAPI's capabilities in constructing type-safe, high-performance APIs with automated documentation demonstrated how modern Python frameworks can provide both developer productivity and runtime performance.
--	--	---

STUDENT-TRAINEE	HTE HEAD/SUPERVISOR
JOSHUA D. BARCOMA (signature over printed name)	 MARLITO DUNGOG (signature over printed name)

Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

vector search postgres

memories

- memory
 - marito's birthday is dec 8, 1984
 - marito's blood type is o+
 - marito loves kare...

Marito Dúngu

Who can see what you share here: Don't show again

FILE EDIT SELECTION VIEW GO RUN TERMINAL HELP

memory_v2

```

49 app.add_middleware(
50     allow_origins=[
51         "http://localhost:3001",
52         "http://localhost:3003",
53         "http://localhost:3090",
54         "http://127.0.0.1:3080",
55         "http://127.0.0.1:3001",
56         "http://127.0.0.1:3003",
57         "http://127.0.0.1:3090"
58     ],
59     # LibreChat frontend
60 ),
61 allow_credentials=True,
62 allow_methods=["*"],
63 allow_headers=["*"],
64 )
65
66 def _row_to_schema(row: dict) -> MemoryRecord:
67     return MemoryRecord(
68         id=row["id"],
69         storage_id=row["storage_id"],
70         storage_name=row["storage_name"],
71         content=row["content"],
72         metadata=row.get("metadata") or {},
73         created_at=row["created_at"],
74         similarity=float(row.get("similarity", 1.0)),
75     )
76
77 def get_memory_service(settings: Settings = Depends(get_settings)) -> MemoryService:
78     service = MemoryService = app.state.memory_service
79     return service
80
81 def get_database() -> Database:
82     return app.state.database
83
84
85
86
87

```

CHAT CODEX

SESSIONS

Understanding API Terminology in Context

Completed in 0s

2 days ago

paalis na netong go to forms na nasa head...

Completed

1 mo ago

can you create a new folder for backend s...

Completed

1 mo ago

Describe what to build next

23/02/2026

LibreChat v0.8.2 - Every AI for Everyone. Privacy policy Terms of service

Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

